[Projeto 03] Ficha técnica

Risco Relativo

▼ **Caso**

Num contexto recente, a descida das taxas de juro no mercado desencadeou um aumento notável na demanda por pedidos de crédito. Os clientes viram este movimento do mercado, como uma oportunidade favorável para financiar grandes compras ou consolidar dívidas existentes, o que elevou o fluxo de pedidos de empréstimo no banco "Super Caja". A equipe de análise de crédito do banco enfrenta um fardo esmagador devido à análise manual necessária para cada solicitação de empréstimo de clientes individuais. Esta metodologia manual resultou num processo ineficiente e atrasado, que afetou negativamente a eficiência e a velocidade com que os pedidos de empréstimo são processados. A situação tornou-se mais crítica devido à preocupação crescente com a taxa de inadimplência, um problema que afeta cada vez mais o setor financeiro, e aumenta a pressão sobre os bancos para identificar e mitigar riscos associados ao crédito.

Para enfrentar esse desafio, a proposta do banco é a automação do processo de análise de crédito usando técnicas de análise avançadas de dados, com o objetivo de melhorar a eficiência, precisão e rapidez na avaliação de pedidos de crédito. Além disso, o banco já tem uma métrica para identificar clientes com pagamento atrasado, o que poderia ser uma ferramenta valiosa para integrar na classificação de risco no novo sistema automatizado.

O objetivo da análise é **identificar o perfil de clientes com risco de inadimplência, montar uma pontuação de crédito através da análise de dados e avaliar o risco relativo, possibilitando assim, classificar os clientes e futuros clientes em diferentes categorias de risco com base na sua probabilidade de inadimplência.** Esta classificação permitirá ao banco tomar

decisões informadas sobre a quem conceder crédito, reduzindo assim o risco de empréstimos não reembolsáveis. Além disso, a integração destas métricas fortalecerá a capacidade do modelo de identificar riscos, contribuindo para a solidez financeira e a eficiência operacional do Banco.

Insumos

Este conjunto de dados contém dados sobre empréstimos concedidos a um grupo de clientes do banco. Os dados estão divididos em 4 tabelas, a primeira com dados do usuário/cliente, a segunda com dados do tipo empréstimo, a terceira com o comportamento de pagamento desses empréstimos, e a quarta com a informação dos clientes já identificados como inadimplentes.

O conjunto de dados está disponível para download neste link **conjunto de dados**. Observe que é um arquivo compactado, você terá que descompactá-lo para acessar os arquivos com as tabelas.

Abaixo, você pode consultar a descrição das variáveis que compõem as tabelas deste conjunto de dados:

Arquivo	Variável	Descrição
user_info	user id	Número de identificação do cliente (único para cada cliente)
	age	Idade do cliente
	sex	Gênero do cliente
	last month salary	Último salário mensal que o cliente informou ao banco
	number dependents	Número de dependentes
loans_outstanding	loan id	Número de identificação do empréstimo (único para cada empréstimo)
	user id	Número de identificação do cliente
	loan type	Tipo de empréstimo (real estate = imóveis, others= outros)
loans_detail	user id	Número de identificação do cliente

	more 90 days overdue	Número de vezes que o cliente apresentou atraso superior a 90 dias
	using lines not secured personal assets	Quanto o cliente está utilizando em relação ao seu limite de crédito, em linhas que não são garantidas por bens pessoais, como imóveis e automóveis
	number times delayed payment loan 30 59 days	Número de vezes que o cliente atrasou o pagamento de um empréstimo (entre 30 e 59 dias)
	debt ratio	Relação entre dívidas e ativos do cliente. Taxa de endividamento = Dívidas / Patrimônio
	number times delayed payment loan 60 89 days	Número de vezes que o cliente atrasou o pagamento de um empréstimo (entre 60 e 89 dias)

default	user id	Número de identificação do cliente
	default flag	Classificação dos clientes inadimplentes (1 para clientes já registrados alguma vez como inadimplentes, 0 para clientes sem histórico de inadimplência)

Objetivo: Realizar uma análise de um conjunto de dados de um Banco através do risco relativo.

Responsável: Andresa Araújo

Ferramentas e Tecnologias: BigQuery, Google Colab, Looker

Processamento e preparo do banco de dados

Conectar/importar dados para ferramentas

Criei as tabelas com os nomes originais: default, loans_detail, loans_outstanding, user_info.

Ambas tem 36 mil linhas/valores/clientes, com exceção da loans_outstanding (são 305.335 empréstimos).

Também notei que a id do cliente varia entre 3 a 5 algorismos.

Identificar e tratar valores nulos

▼ Tabela default: nenhum nulo encontrado

```
SELECT  *
FROM `projeto-03-467414.dataset_03.default`
WHERE default_flag IS NULL
```

▼ Tabela loans_detail: nenhum nulo encontrado

```
SELECT *
FROM `projeto-03-467414.dataset_03.loans_detail`
WHERE user_id IS NULL
  OR more_90_days_overdue IS NULL
  OR using_lines_not_secured_personal_assets IS NULL
  OR number_times_delayed_payment_loan_30_59_days IS NULL
  OR debt_ratio IS NULL
  OR number_times_delayed_payment_loan_60_89_days IS NULL;
```

▼ Tabela loans_outstanding: nenhum nulo encontrado

```
SELECT
*
FROM `projeto-03-467414.dataset_03.loans_outstanding`
WHERE loan_id IS NULL
OR user_id IS NULL
OR loan_type IS NULL
```

▼ Tabela user_info: 7199 valores nulos em last_month_salary e 943 em number_dependents

```
SELECT * FROM `projeto-03-467414.dataset_03.user_info`
WHERE user_id IS NULL
OR age IS NULL
OR sex IS NULL
```

```
OR last_month_salary IS NULL
OR number_dependents IS NULL
```

Ao identificar os nulos, eu rodei queries separadas:

```
SELECT * FROM `projeto-03-467414.dataset_03.user_info`
WHERE last_month_salary IS NULL
```

```
SELECT * FROM `projeto-03-467414.dataset_03.user_info`
WHERE number_dependents IS NULL
```

Levei a informação default_flag para a tabela user_info e criei a view user_info_default:

```
SELECT
u.*,
d.default_flag

FROM projeto-03-467414.dataset_03.user_info u
LEFT JOIN projeto-03-467414.dataset_03.default d
ON
u.user_id = d.user_id
```

Para praticar o comando COALESCE, usei:

```
SELECT
  user_id,
  COALESCE(CAST(last_month_salary AS STRING), CAST(default_flag AS
STRING), CAST(number_dependents AS STRING)) AS user_valor,
  CASE
    WHEN last_month_salary IS NOT NULL THEN 'last_month_salary'
    WHEN default_flag IS NOT NULL THEN 'default_flag'
    WHEN number_dependents IS NOT NULL THEN 'number_dependents'
    ELSE 'Nenhum disponível'
```

```
  END AS origem_valor
FROM `projeto-03-467414.dataset_03.user_info_default`;
```

Decidi criar a primeira versão da tabela user_info_tratada unindo as views:

```
CREATE TABLE projeto-03-467414.dataset_03.user_info_tratada AS
SELECT
ui.age,
ui.sex,
udc.*
FROM projeto-03-467414.dataset_03.user_info_default ui
LEFT JOIN projeto-03-467414.dataset_03.user_info_default_coalesce udc
ON ui.user_id = udc.user_id
```

Identificar e tratar valores duplicados

- Tabela default: nenhum duplicado encontrado

```
SELECT
user_id,
COUNT(*) AS quantidade
FROM `projeto-03-467414.dataset_03.default`
GROUP BY user_id
HAVING
COUNT(*)>1
```

```
SELECT
user_id,
default_flag,
COUNT(*) AS quantidade
FROM `projeto-03-467414.dataset_03.default`
GROUP BY user_id, default_flag
HAVING
COUNT(*)>1
```

▼ Tabela loans_detail: nenhum duplicado encontrado

```
SELECT
user_id,
COUNT(*) AS quantidade
FROM `projeto-03-467414.dataset_03.loans_detail`
GROUP BY user_id
HAVING COUNT(*)>1
```

▼ Tabela loans_outstanding: nenhum duplicado encontrado*

Encontrei 34.510 user_id repetidos, mas é natural pois a mesma pessoa pode solicitar empréstimo mais de uma vez. Mas imagino que vou precisar tratar esses valores, agrupando.

```
SELECT
user_id,
COUNT(*) AS quantidade
FROM projeto-03-467414.dataset_03.loans_outstanding
GROUP BY user_id
HAVING
COUNT(*)>1
```

Para isso, decidi criar logo a variável loans_by_user e salvei numa view:

```
SELECT
  user_id,
  COUNT(loan_id) AS loans_by_user
FROM projeto-03-467414.dataset_03.loans_outstanding
GROUP BY user_id
```

Decidi buscar loan_ids repetidos, pois essa variável sim é crítica nessa tabela. Nenhum duplicado.

```
SELECT
loan_id,
COUNT(*) AS quantidade
FROM projeto-03-467414.dataset_03.loans_outstanding
GROUP BY loan_id
HAVING
COUNT(*)>1
```

▼ Tabela user_info: nenhum duplicado encontrado

```
SELECT
user_id,
COUNT(*) AS quantidade
FROM projeto-03-467414.dataset_03.user_info
GROUP BY user_id
HAVING
COUNT(*)>1
```

Identificar e gerenciar dados fora do escopo da análise

▼ Tabela loans_detail: achei 3 variáveis correlatas. Apliquei o desvio padrão das 3, vi que a que tinha maior desvio era a number_times_delayed_payment_loan_30_59_days (4.14) e salvei numa view.

Pretendo excluir da análise somente a number_times_delayed_payment_loan_60_89_days. Afinal, desvio padrão pequeno indica pouca variabilidade → ou seja, essa variável muda pouco entre os clientes. Pode ser um sinal de que ela não contribui tanto para separar os bons dos maus pagadores.

```
SELECT
CORR(more_90_days_overdue, number_times_delayed_payment_loan_30_
59_days) AS corr_1,
CORR(more_90_days_overdue, number_times_delayed_payment_loan_60_
89_days) AS corr_2,
CORR(number_times_delayed_payment_loan_30_59_days,number_times_d
```

```
elayed_payment_loan_60_89_days) AS corr_3,
STDDEV(more_90_days_overdue) AS dp_more_90,
STDDEV(number_times_delayed_payment_loan_30_59_days) AS dp_30_5
9,
STDDEV(number_times_delayed_payment_loan_60_89_days) AS dp_60_89
FROM `projeto-03-467414.dataset_03.loans_detail`
```

Aproveitei para renomear algumas variáveis com nome muito longo por meio de CTE e criação da tabela limpa

```
CREATE TABLE projeto-03-467414.dataset_03.loans_detail_limpo AS

WITH dados AS (
SELECT
*
FROM `projeto-03-467414.dataset_03.loans_detail`
)
SELECT
* EXCEPT (using_lines_not_secured_personal_assets,number_times_delay
ed_payment_loan_30_59_days, number_times_delayed_payment_loan_60_
89_days),
using_lines_not_secured_personal_assets AS unsec_use,
number_times_delayed_payment_loan_30_59_days AS delay_30_59
FROM projeto-03-467414.dataset_03.loans_detail
```

▼ Tabela user_info: eliminei a variável sex da minha análise por ser discriminatória para o objetivo do projeto e atualizei a tabela manipulada

```
SELECT * EXCEPT (age, sex)
FROM projeto-03-467414.dataset_03.user_info_tratada
```

Identificar e tratar dados discrepantes em variáveis categóricas

▼ Tabela loans_outstanding: ajustei a variável string loan_type deixando tudo em lowercase, corrigi o “others” e criei uma v1 de tabela limpa

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.loans_outstan
ding_limpo AS
WITH dados AS (
  SELECT *, LOWER(loan_type) AS loan_type_limpo
  FROM projeto-03-467414.dataset_03.loans_outstanding
)
SELECT * EXCEPT (loan_type)
FROM dados;
```

Identificar e tratar dados discrepantes em variáveis numéricas

- Tabela default

  Achei que na default_flag, corria o risco de ter alguma linha com erro em que o valor não fosse 0 ou 1, então procurei dessa forma (mas não encontrei nada):

```
SELECT *
FROM projeto-03-467414.dataset_03.default
WHERE default_flag NOT IN (0, 1);
```

- Tabela loans_detail

  Eu quis puxar valores extremos, rodei o comando para todas as variáveis. Achei um outlier em debt ratio. O maior valor da amostra é de 60.212, mas há um valor de 307.001, indicando extremo endividamento.

```
SELECT *
FROM projeto-03-467414.dataset_03.loans_detail_limpo
ORDER BY debt_ratio DESC
LIMIT 20;
```

- Tabela loans_outstanding: sem dados discrepantes
- Tabela user_info

Ao verificar a variável age, identifiquei 3 idades muito extremas.

```
SELECT
*
FROM `projeto-03-467414.dataset_03.user_info_limpo`
ORDER BY age DESC
LIMIT 20;
```

Resolvi substituir pela mediana. Primeiro calculei ela, entre os não-outliers, que deu 52 anos:

```
SELECT
  PERCENTILE_CONT(age, 0.5) OVER() AS median_age
FROM `projeto-03-467414.dataset_03.user_info_limpo`
WHERE age <= 100
LIMIT 1;
```

Depois, substitui os valores extremos por 52 em uma nova coluna:

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.user_info_lim
po AS
SELECT
  *,
  IF(age IS NULL OR age > 100 OR age < 0, 52, age) AS age_limpo
FROM `projeto-03-467414.dataset_03.user_info_limpo`;
```

Criar novas variáveis

▼ Tabela loans_detail

Decidi criar uma variável que soma os atrasos:

```
SELECT
*,
more_90_days_overdue + delay_30_59 AS total_atrasos
```

```
FROM `projeto-03-467414.dataset_03.loans_detail_limpo`
ORDER BY total_atrasos DESC
```

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.loans_detail_li
mpo AS
SELECT
t.total_atrasos,
l.*
FROM projeto-03-467414.dataset_03.total_atrasos t
LEFT JOIN projeto-03-467414.dataset_03.loans_detail_limpo l
ON t.user_id = l.user_id
```

▼ Tabela loans_outstanding

Decidi aproveitar a variável de loan_id agrupado que criei antes para adicionar à tabela.

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.loans_outstan
ding_limpo AS
SELECT
lol.*,
lbu.loans_by_user,
FROM projeto-03-467414.dataset_03.loans_by_user lbu
LEFT JOIN projeto-03-467414.dataset_03.loans_outstanding_limpo lol
ON
lol.user_id = lbu.user_id
```

Unir tabelas

Como a tabela de clientes é a “mais completa”, no sentido de que as outras são dados sobre seus empréstimos, decido fazer um LEFT JOIN pra ela.

Mas primeiro, precisei deixar todas com o mesmo número de linhas (ids dos clientes). Rodei a seguinte query na tabela loans_outstanding.

▼ Códigos:

```
SELECT
  u.user_id,
  COUNT(DISTINCT l.loan_id) AS total_loans,
  IFNULL(
    ARRAY_AGG(DISTINCT l.loan_type_corrigido IGNORE NULLS LIMIT 1)[O
FFSET(0)],
    '0'
  ) AS main_loan_type
FROM
  `projeto-03-467414.dataset_03.user_info_limpo` u
LEFT JOIN
  `projeto-03-467414.dataset_03.loans_outstanding_limpo` l
ON
  u.user_id = l.user_id
GROUP BY
  u.user_id
```

Feito isso, uni as tabelas:

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.uniao_das_tab
elas AS
SELECT
  u.* (EXCEPT age),
  d.* (EXCEPT user_id),
  ld.* (EXCEPT user_id),
  lo.* (EXCEPT user_id),
FROM
  projeto-03-467414.dataset_03.user_info_limpo u
LEFT JOIN
  projeto-03-467414.dataset_03.default d ON u.user_id = d.user_id
LEFT JOIN
  projeto-03-467414.dataset_03.loans_detail_limpo ld ON u.user_id = ld.us
er_id
LEFT JOIN
```

```
 projeto-03-467414.dataset_03.loans_outstanding_limpo_copy lo ON u.us
er_id = lo.user_id
```

Construir tabelas auxiliares

Eu criei tabelas temporárias antes, agregando os valores antes da união.

Análise exploratória

Agrupar e ver dados de acordo com variáveis categóricas

No Looker Studio, vi as infos de idade, tipo de empréstimo e status (adimplente ou inadimplente). Vi que:

- A minoria dos clientes estão classificados como inadimplentes (1,8%). Isso significa que temos alguma mostra validada/classificada pelo banco, o que servirá de referência.
- A maioria dos clientes faz empréstimos para bens diversos, a minoria não solicitou nenhum tipo de empréstimo.

Aplicar medidas de tendência central (moda, média, mediana)

- Notei que a idade dos clientes é bem distribuída, pois a média e a mediana coincidiram (52 anos). O cliente com idade de 36 anos é até mais frequente, mas ainda assim os mais velhos estão em quantidade suficiente para puxar a amostra para cima.
- Em relação aos salários, vi que a média é de $6.420, tendo como valor máximo $1.560.100, o que podemos considerar outlier mas notei outros valores próximos então não diria que é discrepante. A mediana é de $5400, decidi usar ela como substituto para os valores nulos.

Ver distribuição

- Fiz um boxplot de idade, usando como dimensão a variavel de default_flag que informa quem é inadimplente e adimplente. Os inadimplentes tendem a ser mais jovens.

- Também fiz mais um boxplot com o número de dependentes. O grupo de clientes inadimplentes tende a ter um número de dependentes ligeiramente maior, o que pode indicar uma maior carga financeira. Enquanto a mediana para adimplentes é 0, a mediana para inadimplentes é 1.

Aplicar medidas de dispersão (desvio padrão)

- Calculei o desvio padrão das medidas que eu já tinha começado a analisar, como idade, salário e atrasos. O dp do salário é bem relevante, corroborando com a análise anterior.

Calcular quartis, decis ou percentis

▼ Calculei os quartis para idade e já agrupei com a minha coluna de default_flag_limpo para entender os grupos de maus pagadores.

Ela estava como string, porque eu converti erroneamente, então também adicionei um código pra converter, com ajuda do ChatGPT:

```
WITH dados_tratados AS (
  SELECT
    user_id,
    age_limpo,
    CASE
      WHEN TRIM(LOWER(default_flag_limpo)) = 'inadimplente' THEN 1
      WHEN TRIM(LOWER(default_flag_limpo)) = 'adimplente' THEN 0
      ELSE NULL
    END AS flag_int
  FROM `projeto-03-467414.dataset_03.uniao_das_tabelas`
),

quartis AS (
  SELECT
    user_id,
    age_limpo,
    flag_int,
    NTILE(4) OVER (ORDER BY age_limpo) AS faixa_etaria
  FROM dados_tratados
```

```
  WHERE flag_int IS NOT NULL
)

SELECT
  faixa_etaria,
  COUNT(*) AS total_usuarios,
  SUM(CASE WHEN flag_int = 1 THEN 1 ELSE 0 END) AS total_inadimplente
s,
  SUM(CASE WHEN flag_int = 0 THEN 1 ELSE 0 END) AS total_adimplente
s,
  ROUND(SAFE_DIVIDE(SUM(CASE WHEN flag_int = 1 THEN 1 ELSE 0 EN
D), COUNT(*)), 3) AS proporcao_inadimplentes
FROM quartis
GROUP BY faixa_etaria
ORDER BY proporcao_inadimplentes DESC
```

Row	faixa_etaria	total_usuarios	total_inadimplentes	total_adimplentes	proporcao_inadimplentes
1	1	9000	310	8690	0.034
2	2	9000	202	8798	0.022
3	3	9000	119	8881	0.013
4	4	9000	52	8948	0.006

Ou seja, como eu queria encontrar a faixa etária com maior risco de ser um mau pagador, **entendo que é o quartil 1, com 310 inadimplentes, com proporção de 0,03, o maior. Esse quartil vai de 21 a 41 anos.**

Depois, calculei quartis para as outras variáveis:

last_month_salary

```
SELECT
  user_id,
  IFNULL(last_month_salary, 0) AS last_month_salary_tratado,
  NTILE(4) OVER (ORDER BY IFNULL(last_month_salary, 0)) AS quartil_sala
ry
FROM `projeto-03-467414.dataset_03.user_info`
```

number_dependents

```
SELECT
  user_id,
  IFNULL(number_dependents, 0) AS number_dependents_tratado,
  NTILE(4) OVER (ORDER BY IFNULL(number_dependents, 0)) AS quartil_d
ependents
FROM `projeto-03-467414.dataset_03.user_info`
```

atraso de 90 dias

```
SELECT
more_90_days_overdue,
NTILE(4) OVER (ORDER BY more_90_days_overdue) AS quartis_90_days
FROM `projeto-03-467414.dataset_03.loans_detail_limpo`
```

atraso de 30 a 59 dias

```
SELECT
delay_30_59,
NTILE(4) OVER (ORDER BY delay_30_59) AS quartis_delay_30_59
FROM `projeto-03-467414.dataset_03.loans_detail_limpo`
```

loans

```
SELECT
loans_by_user,
loan_type_corrigido,
NTILE(4) OVER (ORDER BY loans_by_user) AS quartis_loans,
FROM `projeto-03-467414.dataset_03.loans_outstanding_limpo`
```

Calcular correlação entre variáveis numéricas

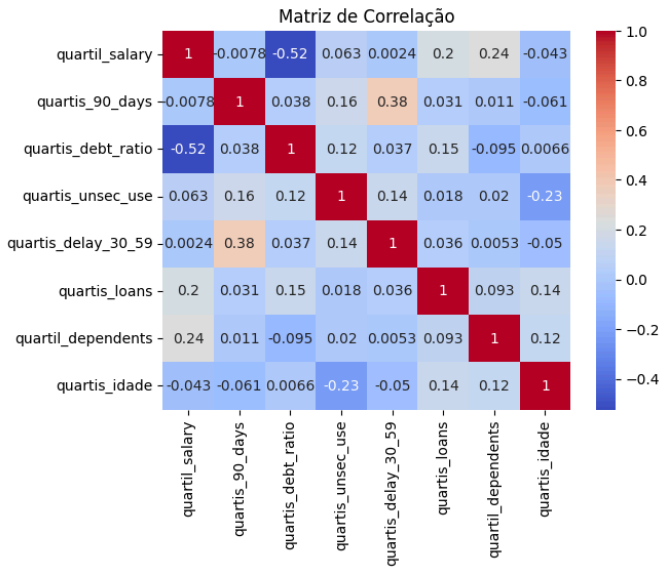
▼ Fiz alguns cruzamentos no BigQuery e, entre as variáveis em si, só a default_flag que apresentou uma correlação (fracamente positiva) com as variáveis de atraso, como atraso de 90 dias, e 30 a 59 dias (entre 0,299 e 0,3 o índice de Pearson). Juntei os quartis em uma view para correlacioná-los também:

```
SELECT
  s.user_id,quartil_salary,
  ld.quartis_90_days, ld.quartis_debt_ratio, ld.quartis_unsec_use, quartis_d
elay_30_59,
  lo.quartis_loans,
  ui.quartil_dependents, ui.quartis_idade
FROM
  `projeto-03-467414.dataset_03.quartil_salary` s
LEFT JOIN `projeto-03-467414.dataset_03.quartis_loans_detail` ld ON s.us
er_id = ld.user_id
LEFT JOIN `projeto-03-467414.dataset_03.quartis_loans_outstanding` lo O
N s.user_id = lo.user_id
LEFT JOIN `projeto-03-467414.dataset_03.quartis_user_info` ui ON s.user
_id = ui.user_id
```

Pra facilitar, levei esse csv para o colab analisar todas as variáveis de uma vez:

```
import pandas as pd

# Suba o arquivo pelo botão de upload no Colab
from google.colab import files
uploaded = files.upload()

# Chame o arquivo
df = pd.read_csv("bq-results-20250805-133204-1754404796246.csv")

variaveis = ['quartil_salary', 'quartis_90_days', 'quartis_debt_ratio', 'quartis
_unsec_use', 'quartis_delay_30_59',
             'quartis_loans','quartil_dependents','quartis_idade']
```

```
display(df[variaveis].corr())

import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df[variaveis].corr(), annot=True, cmap='coolwarm')
plt.title("Matriz de Correlação")
plt.show()
```



Apesar de não ter trazido nenhuma correlação forte, nos diz que:

quartis_debt_ratio × quartil_salary: **-0.52**
Correlação negativa moderada: quanto maior o salário (quartil), menor a razão de endividamento.

quartis_unsec_use × quartis_debt_ratio: **0.37**
Pessoas que usam mais crédito não garantido tendem a estar mais endividadas.

quartis_delay_30_59 × quartis_90_days: **0.38**
Quem atrasa o pagamento por 30-59 dias tem chance maior de atrasar mais de 90 dias.

👉 Correlações fracas, mas com algum sentido:

- quartil_dependents × quartis_debt_ratio: **0.12**

  Ter mais dependentes pode influenciar o endividamento.

- quartis_loans × quartis_delay_30_59: **0.14**

  Um número maior de empréstimos pode aumentar a chance de atrasos.

  Por fim, inclui a variável default_flag (que seria a variável-alvo):

  - quartis_debt_ratio: **0.24**
    - Quanto **maior a razão de endividamento**, **maior a chance de inadimplência**. Faz sentido: quem compromete mais da renda está mais vulnerável.
    - quartis_delay_30_59: **0.23**
      - Pessoas com **histórico de atraso curto** também tendem a se tornar inadimplentes. Pode ser um **sinal inicial de risco**.
    - quartis_90_days: **0.16**
      - Atraso longo também se relaciona com a flag, como esperado.

Aplicar técnica de análise

Calcular risco relativo

▼ **Fator de risco: Idade**

```
WITH tabela AS (
  SELECT
    categorias_idade,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_idade
),
```

```
taxas AS (
  SELECT
    categorias_idade,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_idade = 1 THEN taxa_incidencia END),
      MAX(CASE WHEN categorias_idade = 2 THEN taxa_incidencia END)
    ) AS risco_relativo
  FROM taxas
)
SELECT
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_idade)
CROSS JOIN risco r
ORDER BY categorias_idade;
```

Row	categorias_idade	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	309	8691	9000	0.034333333333...	1.485576923076...
2	2	208	8792	9000	0.023111111111...	1.485576923076...
3	3	116	8884	9000	0.012888888888...	1.485576923076...
4	4	50	8950	9000	0.005555555555...	1.485576923076...

Maior incidência de maus pagadores: Q1 (0,034) - grupo exposto

Menor incidência: Q4 (0,005) - grupo não-exposto

Risco relativo: 6,8. **Há ~6x mais risco de maus pagadores em clientes mais novos.**

▼ **Fator de risco: Atraso de pelo menos 30 dias**

```
WITH tabela AS (
  SELECT
    categorias_delay_30_59,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_delay_30_59
),
taxas AS (
  SELECT
    categorias_delay_30_59,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_delay_30_59 = 1 THEN taxa_incidencia E
ND),
      MAX(CASE WHEN categorias_delay_30_59 = 2 THEN taxa_incidencia E
ND)
    ) AS risco_relativo
  FROM taxas
)
SELECT
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_delay_30_59)
```

```
CROSS JOIN risco r
ORDER BY categorias_delay_30_59;
```

Row	categorias_delay...	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	17	8983	9000	0.001888888888...	0.548387096774...
2	2	31	8969	9000	0.003444444444...	0.548387096774...
3	3	16	8984	9000	0.001777777777...	0.548387096774...
4	4	619	8381	9000	0.068777777777...	0.548387096774...

Maior incidência de maus pagadores: Q4 (0,068) - grupo exposto

Menor incidência: Q3 (0,001) - grupo não-exposto

Risco relativo: 40. H**á 40x mais risco de maus pagadores em clientes com mais atrasos no pagamento de pelo menos 30 dias.**

▼ **Fator de risco: Atraso de mais de 90 dias**

```
WITH tabela AS (
  SELECT
    categorias_more_90_days_overdue,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_more_90_days_overdue
),
taxas AS (
  SELECT
    categorias_more_90_days_overdue,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
```

```
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_more_90_days_overdue = 1 THEN taxa_i
ncidencia END),
      MAX(CASE WHEN categorias_more_90_days_overdue = 2 THEN taxa_i
ncidencia END)
    ) AS risco_relativo
  FROM taxas
)
SELECT
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_more_90_days_overdue)
CROSS JOIN risco r
ORDER BY categorias_more_90_days_overdue;
```

Row	categorias_more...	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	10	8990	9000	0.001111111111...	0.625
2	2	16	8984	9000	0.001777777777...	0.625
3	3	16	8984	9000	0.001777777777...	0.625
4	4	641	8359	9000	0.071222222222...	0.625

Maior incidência de maus pagadores: Q4 (0,071) - grupo exposto

Menor incidência: Q3 (0,001) - grupo não-exposto

Risco relativo: 64. H**á 64x mais risco de maus pagadores em quem atrasa +90 dias.**

▼ **Fator de risco: Quantidade de empréstimos**

```
WITH tabela AS (
  SELECT
    categorias_loans,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_loans
),
taxas AS (
  SELECT
    categorias_loans,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_loans = 1 THEN taxa_incidencia END),
      MAX(CASE WHEN categorias_loans = 2 THEN taxa_incidencia END)
    ) AS risco_relativo
  FROM taxas
)
SELECT
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_loans)
CROSS JOIN risco r
ORDER BY categorias_loans;
```

Row	categorias_loans	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	299	8701	9000	0.033222222222...	1.708571428571...
2	2	175	8825	9000	0.019444444444...	1.708571428571...
3	3	104	8896	9000	0.011555555555...	1.708571428571...
4	4	105	8895	9000	0.011666666666...	1.708571428571...

Maior incidência de maus pagadores: Q1 (0,033) - grupo exposto

Menor incidência: Q3 (0,011) - grupo não-exposto

Risco relativo: 2,8. **Há quase 3x mais risco de maus pagadores em quem tem maior quantidade de empréstimos.**

▼ Fator de risco: Salário

Ao chegar nessa etapa, notei que usei o COALESCE de forma equivocada, associando variáveis muito diferentes (salários e número de dependentes). Portanto, ajustei a tabela criando colunas separadas tratando os dados com as suas medianas.

```
CREATE OR REPLACE TABLE projeto-03-467414.dataset_03.uniao_das_tab
elas AS
SELECT
u.* EXCEPT (salary, dependents),
ui.salary_corrigido,
ui.dependents_corrigido,
FROM projeto-03-467414.dataset_03.uniao_das_tabelas u
LEFT JOIN projeto-03-467414.dataset_03.user_info_limpo ui
ON u.user_id=ui.user_id
```

Agora sim, calculando o fator de risco:

```
WITH tabela AS (
  SELECT
    categorias_salary,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
```

```
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_salary
),
taxas AS (
  SELECT
    categorias_salary,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_salary = 1 THEN taxa_incidencia END),
      MAX(CASE WHEN categorias_salary = 2 THEN taxa_incidencia END)
    ) AS risco_relativo
  FROM taxas
)
SELECT
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_salary)
CROSS JOIN risco r
ORDER BY categorias_salary;
```

Row	categorias_salary	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	268	8732	9000	0.029777777777...	1.472527472527...
2	2	182	8818	9000	0.020222222222...	1.472527472527...
3	3	159	8841	9000	0.017666666666...	1.472527472527...
4	4	74	8926	9000	0.008222222222...	1.472527472527...

Maior incidência de maus pagadores: Q1 (0,029) - grupo exposto

Menor incidência: Q4 (0,008) - grupo não-exposto

Risco relativo: 3,6. **Há mais risco de maus pagadores em quem possui menores salários declarados no último mês.**

▼ **Fator de risco: Nº de dependentes**

```
WITH tabela AS (
  SELECT
    categorias_dependents,
    COUNT(CASE WHEN default_flag_corrigido = 1 THEN 1 END) AS mau_p
agadores,
    COUNT(CASE WHEN default_flag_corrigido = 0 THEN 1 END) AS bom_p
agadores,
    COUNT(*) AS total
  FROM `projeto-03-467414.dataset_03.quartis_categorias`
  GROUP BY categorias_dependents
),
taxas AS (
  SELECT
    categorias_dependents,
    SAFE_DIVIDE(mau_pagadores, total) AS taxa_incidencia
  FROM tabela
),
risco AS (
  SELECT
    SAFE_DIVIDE(
      MAX(CASE WHEN categorias_dependents = 1 THEN taxa_incidencia E
ND),
      MAX(CASE WHEN categorias_dependents = 2 THEN taxa_incidencia E
ND)
    ) AS risco_relativo
  FROM taxas
)
SELECT
```

```
  t.*,
  tx.taxa_incidencia,
  r.risco_relativo
FROM tabela t
JOIN taxas tx USING (categorias_dependents)
CROSS JOIN risco r
ORDER BY categorias_dependents;
```

Row	categorias_depen...	mau_pagadores	bom_pagadores	total	taxa_incidencia	risco_relativo
1	1	155	8845	9000	0.017222222222...	1.131386861313...
2	2	137	8863	9000	0.015222222222...	1.131386861313...
3	3	172	8828	9000	0.019111111111...	1.131386861313...
4	4	219	8781	9000	0.024333333333...	1.131386861313...

Maior incidência de maus pagadores: Q4 (0,024) - grupo exposto

Menor incidência: Q2 (0,015) - grupo não-exposto

Risco relativo: 1,6. **Há um pouco mais de risco de maus pagadores em quem tem maior número de de dependentes.**

Aplicar segmentação por Score

▼ Códigos:

```
WITH dummies AS (
   SELECT
      *,
      -- Fator de risco: idade (Q1)
      CASE WHEN categorias_idade = 1 THEN 1 ELSE 0 END AS idade_r,

      -- Fator de risco: atraso pelo menos 30d (Q4)
      CASE WHEN categorias_delay_30_59 = 4 THEN 1 ELSE 0 END AS atra
so_30d_r,
```

```
        -- Fator de risco: atraso de +90d (Q4)
        CASE WHEN categorias_more_90_days_overdue = 4 THEN 1 ELSE 0 E
ND AS atraso_90d_r,

        -- Fator de risco: salário (Q1)
        CASE WHEN categorias_salary = 1 THEN 1 ELSE 0 END AS salario_r,

        -- Fator de risco: empréstimos (Q1)
        CASE WHEN categorias_loans = 1 THEN 1 ELSE 0 END AS emprestim
o_r,

        -- Fator de risco: dependentes (Q4)
        CASE WHEN categorias_dependents = 4 THEN 1 ELSE 0 END AS dep
endentes_r
    FROM projeto-03-467414.dataset_03.categorias_quartis
),
pontuacao AS (
    SELECT
        *,
        -- Soma dos fatores de risco
        (idade_r + atraso_30d_r + atraso_90d_r + salario_r + emprestimo_r +
dependentes_r) AS pontuacao_risco,

        -- Previsão com base na pontuação
        CASE
            WHEN (idade_r + atraso_30d_r + atraso_90d_r + salario_r + empres
timo_r + dependentes_r) >= 3 THEN 1
            ELSE 0
        END AS previsao
    FROM dummies
),
matriz_confusao AS (
    SELECT
        SUM(CASE WHEN previsao = 1 AND default_flag_corrigido = 1 THEN 1
ELSE 0 END) AS vp,
        SUM(CASE WHEN previsao = 0 AND default_flag_corrigido = 1 THEN
```

```
1 ELSE 0 END) AS fn,
        SUM(CASE WHEN previsao = 1 AND default_flag_corrigido = 0 THEN
1 ELSE 0 END) AS fp,
        SUM(CASE WHEN previsao = 0 AND default_flag_corrigido = 0 THEN
1 ELSE 0 END) AS vn
    FROM pontuacao
)
SELECT
    vp,
    fn,
    fp,
    vn,
    (vp + vn) / (vp + vn + fp + fn) AS acuracia,
    vp / (vp + fp) AS precisao,
    vp / (vp + fn) AS recall,
    2 * ((vp / (vp + fp)) * (vp / (vp + fn))) / ((vp / (vp + fp)) + (vp / (vp + fn)))
AS f1_score
FROM matriz_confusao;
```

Row	vp	fn	fp	vn	acuracia	precisao	recall	f1_score
1	541	142	6294	29023	0.821222222222...	0.079151426481...	0.792093704245...	0.143921255653...

Acurácia ✅ Recall ✅

Precisão ❌ F1-Score ❌

Esse modelo 1 é bom em identificar os riscos (**alto recall**), mas é muito ruim em ser preciso (**baixa precisão**). A **alta acurácia** é uma ilusão causada pela baixa frequência de "maus pagadores" no conjunto de dados.

Decido criar um modelo 2, onde a nota de corte em vez de 3 será 5. Quero ver se o modelo se torna mais conservador e reduz a quantidade de Falsos Positivos.

Row	vp	fn	fp	vn	acuracia	precisao	recall	f1_score
1	130	553	322	34995	0.975694444444...	0.287610619469...	0.190336749633...	0.229074889867...

Apesar de algumas melhorias, ainda continuo com problemas na precisão; impactando no f1 score geral.

Como nosso objetivo é minimizar o risco de crédito no Super Caja e capturar o máximo de maus pagadores possível (priorizando o Recall), decido ficar com o modelo 1, mesmo com a precisão baixa.

[Marco 2] Regressão Logística com o Google Colab

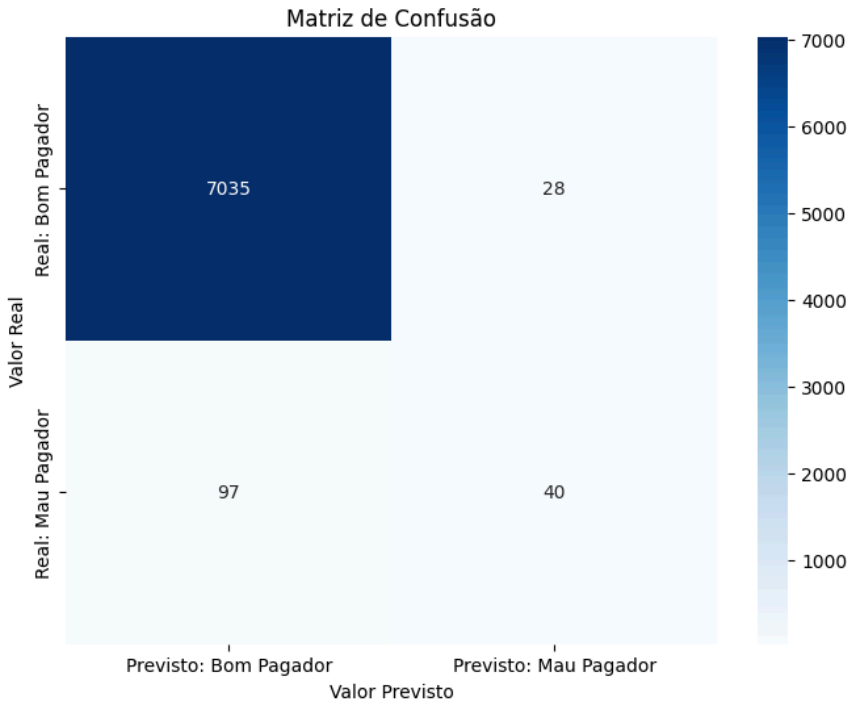
▼ Conectei os dados via arquivo csv e rodei o seguinte código:

```
# Passo 1: Importar as bibliotecas necessárias
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_
score, recall_score, f1_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Passo 2: Carregar os dados a partir do arquivo CSV
# Substitua 'nome_do_seu_arquivo.csv' pelo nome do arquivo que você fe
z o upload
nome_arquivo = 'bquxjob_731196da_198ba50cea2.csv'  # ←—— COLOCAR
ENTRE ASPAS
df = pd.read_csv(nome_arquivo) # ←—— USAR A VARIÁVEL AQUI

# Exibir as primeiras linhas para verificar
print("Dados carregados do CSV:")
print(df.head())

# Passo 3: Preparar os Dados (Dummies)
# A partir daqui, o código é o mesmo do passo a passo anterior
variaveis_categoricas = [
    'categorias_idade',
    'categorias_delay_30_59',
    'categorias_more_90_days_overdue',
```

```
    'categorias_salary',
    'categorias_loans',
    'categorias_dependents'
]

# Converter as colunas categóricas em variáveis dummy
df_dummies = pd.get_dummies(df, columns=variaveis_categoricas, drop_f
irst=True, dtype=int)

print("\nDataFrame com variáveis dummy:")
print(df_dummies.head())

# Passo 4: Definir Variáveis e Dividir os Dados
y = df_dummies['default_flag_corrigido']
X = df_dummies.drop('default_flag_corrigido', axis=1)

X_treino, X_teste, y_treino, y_teste = train_test_split(X, y, test_size=0.2, ran
dom_state=42, stratify=y)

print(f"\nTamanho do conjunto de treino: {X_treino.shape[0]} amostras")
print(f"Tamanho do conjunto de teste: {X_teste.shape[0]} amostras")

# Passo 5: Treinar o Modelo de Regressão Logística
modelo = LogisticRegression(solver='liblinear', random_state=42)
modelo.fit(X_treino, y_treino)

print("\nModelo de Regressão Logística treinado com sucesso!")

# Passo 6: Fazer Previsões e Avaliar o Modelo
previsoes = modelo.predict(X_teste)

print("\n--- Relatório de Métricas do Modelo ---")
print(f"Acurácia: {accuracy_score(y_teste, previsoes):.4f}")
print(f"Precisão: {precision_score(y_teste, previsoes):.4f}")
print(f"Recall: {recall_score(y_teste, previsoes):.4f}")
print(f"F1-Score: {f1_score(y_teste, previsoes):.4f}")
```

```
print("\n--- Relatório de Classificação Completo ---")
print(classification_report(y_teste, previsoes))

# Visualizar a matriz de confusão
matriz = confusion_matrix(y_teste, previsoes)
plt.figure(figsize=(8, 6))
sns.heatmap(matriz, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Previsto: Bom Pagador', 'Previsto: Mau Pagador'],
            yticklabels=['Real: Bom Pagador', 'Real: Mau Pagador'])
plt.title('Matriz de Confusão')
plt.ylabel('Valor Real')
plt.xlabel('Valor Previsto')
plt.show()
```

```
--- Relatório de Classificação Completo ---
              precision    recall  f1-score   support

           0       0.99      1.00      0.99      7063
           1       0.59      0.29      0.39       137

    accuracy                           0.98      7200
   macro avg       0.79      0.64      0.69      7200
weighted avg       0.98      0.98      0.98      7200
```



Ou seja, o modelo melhorou.

Para o grupo de bom pagador:

Precisão: 0.99. Quando o modelo prevê que alguém é um bom pagador, ele acerta 99% das vezes. Isso é ótimo.

Recall: 1.00. O modelo identifica todos os bons pagadores (ou quase todos, com 1.00 de recall).

Para o grupo mau pagador:

Precisão: 0.59. Quando o modelo prevê que alguém é um mau pagador, ele acerta 59% das vezes. **Isso é uma melhoria enorme** em relação aos 7% e 28% que você tinha antes.

Recall: 0.29. O modelo consegue identificar 29% dos verdadeiros maus pagadores. Houve uma queda aqui em relação ao seu primeiro modelo, que tinha recall de 79%.

F1-Score (Classe 1): 0.39. O F1-Score subiu de 0.14 para 0.39, o que indica um equilíbrio melhor entre precisão e recall para a classe de interesse (`mau pagador`).

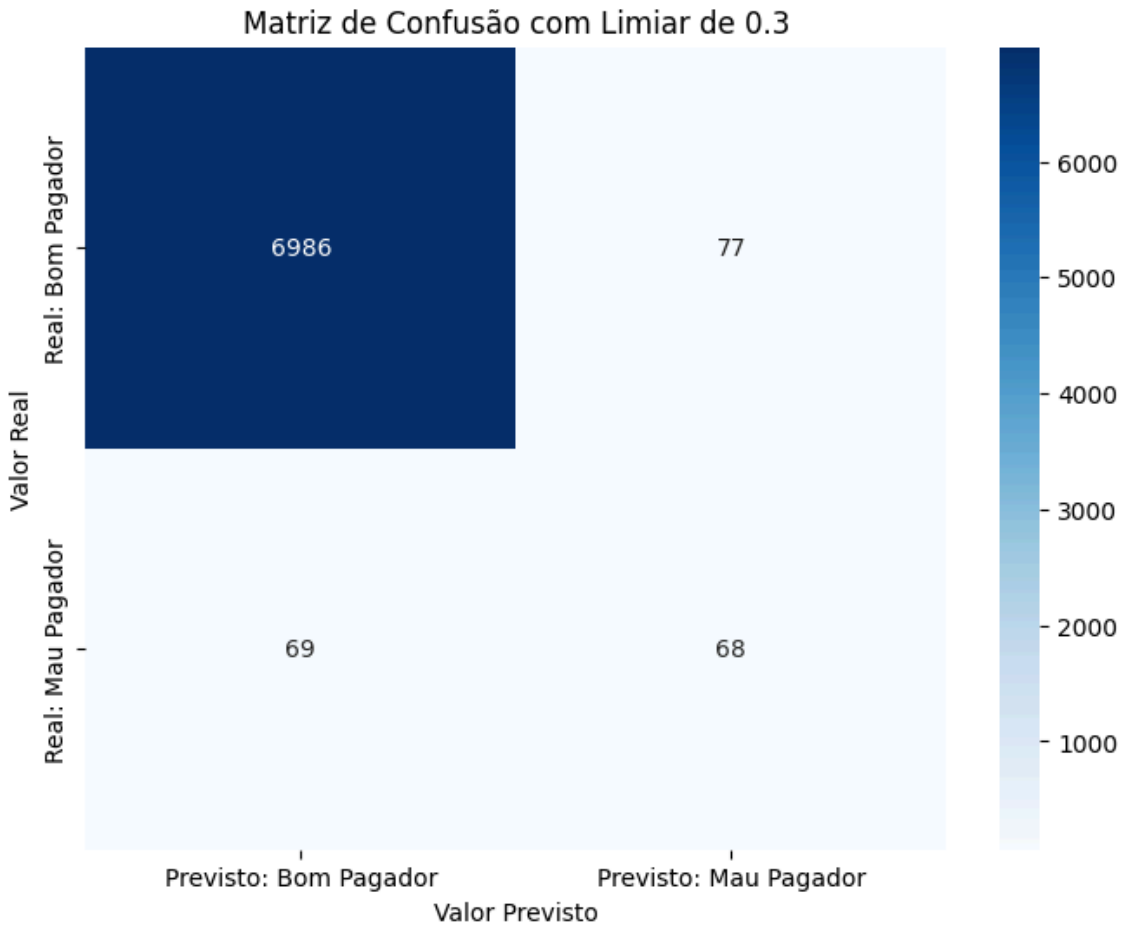
Acurácia Global: 0.98. A acurácia continua alta, mas agora sabemos que ela se deve principalmente ao grande número de bons pagadores.

Entretanto, o **Recall** caiu, agora está em 29%. Isso significa que o modelo está mais cauteloso, mas acaba deixando escapar uma quantidade considerável de "maus pagadores" (97 deles, para ser exato, como visto na matriz de confusão).

Para equilibrar melhor o modelo, decido abrir mão da precisão para aumentar um pouco o recall por meio do ajuste no limiar de decisão (predict()). Nesse modelo, se a probabilidade de um cliente ser um mau pagador for maior que 0.5, o modelo o classifica como 1 (mau pagador); caso contrário, o classifica como 0 (bom pagador). **Então reduzo para 0.3 pois meu objetivo é sempre captar os maus pagadores:**

```
# Passo 6 (Modificado): Fazer Previsões e Avaliar o Modelo com Limiar
Ajustado

# Obter as probabilidades para cada classe (0 e 1)
# O modelo retorna uma matriz onde a primeira coluna é a probabilidad
e da classe 0
# e a segunda coluna é a probabilidade da classe 1.
probabilidades = modelo.predict_proba(X_teste)

# Extrair as probabilidades para a classe de interesse (classe 1 - mau p
agador)
probabilidades_classe_1 = probabilidades[:, 1]

# Definir um novo limiar de decisão
# Vamos começar com 0.3 para tentar aumentar o Recall.
# Você pode testar outros valores como 0.2, 0.4, etc.
limiar = 0.3

# Aplicar o novo limiar para gerar as previsões ajustadas
previsoes_ajustadas = (probabilidades_classe_1 >= limiar).astype(int)

# Calcular e exibir as métricas de desempenho com o novo limiar
```

```
print(f"\n--- Relatório de Métricas com Limiar de {limiar} ---")
print(f"Acurácia: {accuracy_score(y_teste, previsoes_ajustadas):.4f}")
print(f"Precisão: {precision_score(y_teste, previsoes_ajustadas):.4f}")
print(f"Recall: {recall_score(y_teste, previsoes_ajustadas):.4f}")
print(f"F1-Score: {f1_score(y_teste, previsoes_ajustadas):.4f}")
print("\n--- Relatório de Classificação Completo ---")
print(classification_report(y_teste, previsoes_ajustadas))

# Visualizar a matriz de confusão com o novo limiar
matriz_ajustada = confusion_matrix(y_teste, previsoes_ajustadas)
plt.figure(figsize=(8, 6))
sns.heatmap(matriz_ajustada, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Previsto: Bom Pagador', 'Previsto: Mau Pagador'],
            yticklabels=['Real: Bom Pagador', 'Real: Mau Pagador'])
plt.title(f'Matriz de Confusão com Limiar de {limiar}')
plt.ylabel('Valor Real')
plt.xlabel('Valor Previsto')
plt.show()
```



Performance

- **Acurácia:** Permanece alta, em 0.9797
- **Precisão:** Caiu de 0.59 para **0.47.** Isso é a "troca" que acontece: para capturar mais maus pagadores, a confiança em cada previsão de risco diminui.
- **Recall:** Subiu de 0.29 para **0.50**.

A versão anterior (limiar 0.5) tinha uma precisão maior, mas era ineficaz em encontrar os maus pagadores.

- **F1-Score:** Subiu de 0.39 para **0.48**.

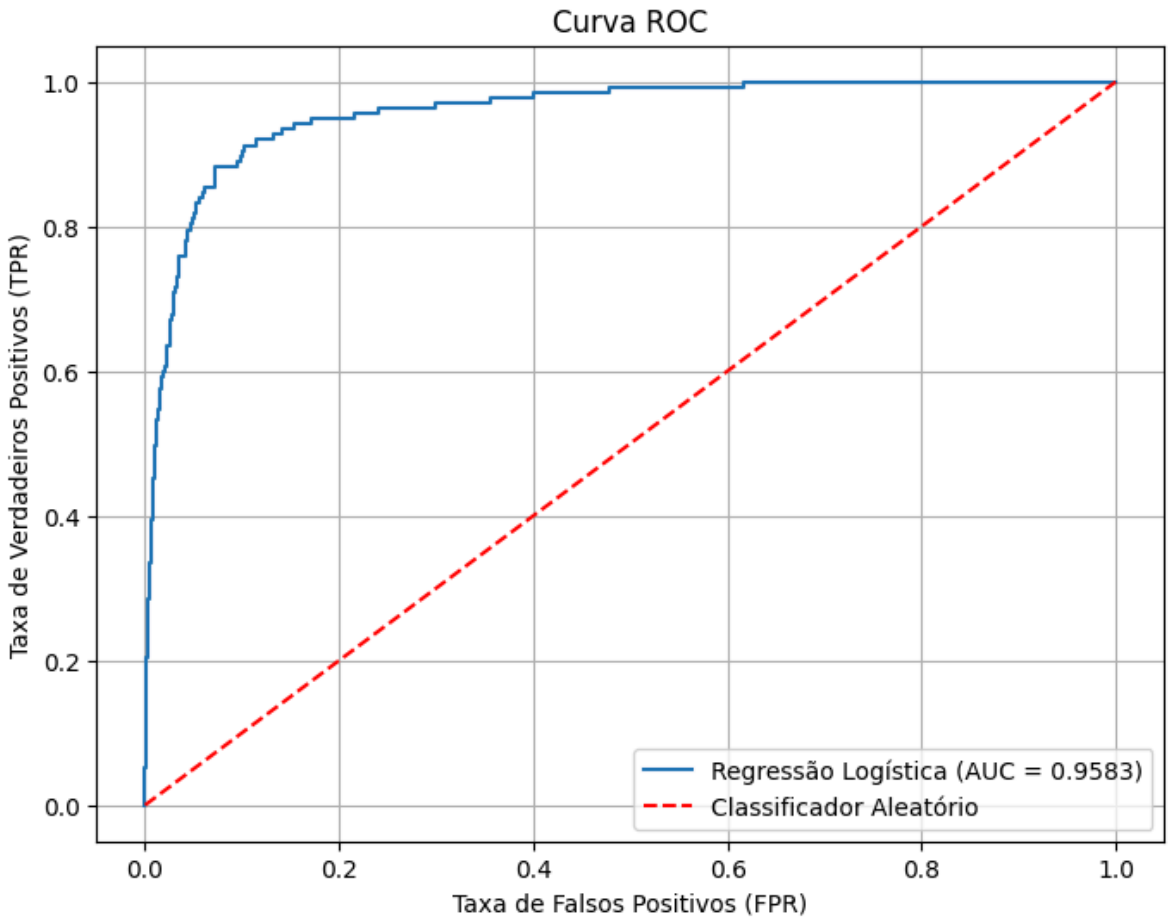
O modelo atual é o melhor porque tem uma precisão razoável (47%): de cada 100 alertas de "mau pagador", quase metade são corretos. E agora

temos um Recall mais alto (50%): capturando metade dos verdadeiros maus pagadores.

Aproveitei para gerar a curva ROC também

```
# Importar as bibliotecas necessárias para a curva ROC
from sklearn.metrics import roc_curve, roc_auc_score

# Passo 7: Gerar a Curva ROC e o AUC

# Obter as probabilidades para a classe positiva (classe 1)
# Usaremos as mesmas probabilidades que você usou para ajustar o li
miar
probabilidades = modelo.predict_proba(X_teste)
probabilidades_classe_1 = probabilidades[:, 1]

# Calcular a Curva ROC
# roc_curve retorna a taxa de falsos positivos (fpr), a taxa de verdadeir
os positivos (tpr)
# e os limiares.
fpr, tpr, thresholds = roc_curve(y_teste, probabilidades_classe_1)

# Calcular a Área sob a Curva (AUC)
auc_score = roc_auc_score(y_teste, probabilidades_classe_1)

# Plotar a Curva ROC
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Regressão Logística (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], 'r--', label='Classificador Aleatório') # Linha de bas
e aleatória
plt.xlabel('Taxa de Falsos Positivos (FPR)')
plt.ylabel('Taxa de Verdadeiros Positivos (TPR)')
plt.title('Curva ROC')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```

```
print(f"\nÁrea sob a Curva (AUC): {auc_score:.4f}")
```



O eixo Y atinge valores altos, indicando que o algoritmo consegue captar muitos casos de inadimplentes. O eixo X são os “falsos alarmes”, ele ainda tá baixo quando os valores no Y sobem. A linha vermelha seria o palpite/acaso, que felizmente está bem distante do modelo. Por fim, o AUC foi de 0,9, muito próximo de 1, o que indica um modelo muito bom.

Próximos passos

- O modelo é bem desbalanceado, 137 maus pagadores em 7200 amostras, então eu precisaria estudar e praticar outras técnicas para lidar com isso.

- A base tinha ~20% de valores desconhecidos, o que pode ter comprometido o treinamento do algoritmo pois aquele valor da mediana talvez não refletia a realidade.
- Criar novas variáveis para aprimorar as categorias.
- Testar outros modelos, além da regressão.

Links de interesse

📊 Dashboard no Looker Studio

👩‍💻 Notebook no Google Colab

💻 Apresentação